

第9章 Shell 编程





学习shell编程的目的:

在需要完成大量复杂、重复性的工作时，不需要在命令行重复执行命令，直接运行shell脚本即可，大大的节省了时间提高了效率。

类型	触发场景	加载配置文件	典型用途
登录Shell	用户登录、 bash -- login	/etc/profile , ~/.profile	交互式用户环境初始化
非交互式Shell	执行脚本、 bash -c "cmd"	不加载 (除非Shebang指定)	自动化脚本运行



例如：批量修改文件名

```
# touch article_{1..50}.html （基础批量创建方法）  
# ls  
# article_1.html article_2.html article_3.html .....
```

```
#!/bin/bash
```

```
for file in $(find . -maxdepth 1 -name "*.html"); do  
    mv "$file" "test_${file#*_}"  
done
```



Shell 编程概述

- Shell脚本 (Shell script)
 - 基于Shell语言的语法规则编写而成，支持Linux/UNIX下的命令的调用。
- Shell语言是一种解释型语言
 - 不需要经过编译、汇编等过程。
- Shell脚本扩展名为 “*.sh” 。
 - 本身是一个文本文件。
- Bash是Ubuntu下默认的Shell解释程序。



shell程序文件的建立实例:

❑ 用vim编辑器建立shell程序文件test1的步骤:

- ① 键入命令“**vim test1**”，启动vim;
- ② 按命令“**i**”，进入vim的插入状态;
- ③ 输入程序**文件内容**;

```
#!/bin/bash
```

```
clear
```

```
cal 1 98
```

```
cal 2 98
```

```
cal 3 98
```

- ④ 按<**Esc**>键，再键入“**:**”，切换到vim的命令状态;
- ⑤ 在“**:**”后键入命令“**wq**”，即保存文件内容后，退出vim。



在一般命令状态下，键入以下命令：

- | | |
|---------------------|------------------------------------|
| q | 结束编辑（ <u>打开的文档未作任何修改</u> ），退出vim |
| q! | 放弃编辑不作保存（ <u>打开的文档已编辑过</u> ），退出vim |
| wq | 保存当前文件后退出vim |
| wq! | 强制性保存当前文件（包括只读文件）后退出vim |
| w [filename] | 用于对新建文件（直接键入vim）或当前正编辑的文件作保存后继续编辑 |



shell 脚本

按顺序执行的Linux命令集。shell 脚本按行解释。

□ Shell 脚本的编写

- 1) Shell 脚本是纯文本文件，可以使用任何文本编辑器编写。
- 2) Shell 脚本通常是以 .sh 作为后缀名。

□ Shell 脚本的执行

```
chmod +x script_name  
./script_name
```

```
bash script_name
```



- **方法1:** 为Shell脚本添加可执行权限

```
zp@lab:~/shell$ chmod +x test001.sh
zp@lab:~/shell$ ./test001.sh
Hello, Shell!
zp@lab:~/shell$
zp@lab:~/shell$ /home/zp/shell/test001.sh
Hello, Shell!
```

- **方法2:** 直接使用bash或sh来执行Shell脚本。

```
zp@lab:~/shell$ bash test001.sh
Hello, Shell!
zp@lab:~/shell$ sh test001.sh
Hello, Shell!
```

- **方法3:** 在当前环境使用source执行Shell脚本。

```
zp@lab:~/shell$ source test001.sh
Hello, Shell!
zp@lab:~/shell$ . test001.sh
Hello, Shell!
zp@lab:~/shell$
zp@lab:~/shell$ source ./test001.sh
Hello, Shell!
```




第9章 Shell编程

❑ 将shell程序文件视作命令来执行:

\$ **<shell程序文件名> [<参数1> <参数2>]**

❑ 例:

① \$ **chmod u+x test1** 给程序文件赋予可执行权限

② \$ **/home/ubuntu/test1** 以完整的路径名键入程序名

或 \$ **./test1** 以相对路径名键入程序名

❑ 在shell脚本中一般使用的形参为: \$0、\$1、\$2、.....\$9——**位置形参**

❑ 当程序运行时, 命令行中的<参数1>、<参数2>...依次赋值给\$1、\$2、..., 通常称运行命令中所带的参数为实参

❑ **\$0** 是一个特殊的形参, 其值规定为当前运行的**shell脚本名本身**



第9章 Shell编程

- 脚本内容: ➡ • 运行脚本: ➡ • 输出结果:

```
1  #!/bin/bash
2  echo "脚本名: $0"
3  echo "第一个参数: $1"
4  echo "第二个参数: $2"
5  echo "所有参数: $@"
6  echo "参数个数: $#"
```

```
1  chmod +x script.sh
2  ./script.sh 参数1 参数2
```

```
1  脚本名: ./script.sh
2  第一个参数: 参数1
3  第二个参数: 参数2
4  所有参数: 参数1 参数2
5  参数个数: 2
```



位置参量（命令行参数）

- ❑ 位置参量是一组特殊的内置变量，通常被 shell 脚本用来从命令行接受参数，或被函数用来保存传递给它的参数。
- ❑ 执行 shell 脚本时，用户可以通过命令行向脚本传递信息，跟在脚本名后面的用空格隔开的每个字符串都称为位置参量。
- ❑ 在脚本中使用这些参数时，需通过位置参量来引用。例如：\$1 表示第一个参数，\$2 表示第二个参数，以此类推。\$9 以后需要用花括号把数字括起来，如第 10 个位置参量以 \${10} 的方式来访问。



第9章 Shell 编程

位置参量列表

<code>\$0</code>	当前脚本的文件名
<code>\$1-\$9</code>	第 1 个到第 9 个位置参量
<code>\${10}</code>	第 10 个位置参量，类似地，有 <code>\${11}</code> ， ...
<code>\$#</code>	位置参量的个数
<code>\$*</code>	以单字符串显示所有位置参量
<code>@</code>	未加双引号时与 <code>\$*</code> 含义相同，加双引号时有区别
<code>\$\$</code>	脚本运行的当前进程号
<code>!</code>	最后一个后台运行的进程的进程号
<code>?</code>	显示前面最后一个命令的退出状态。 0 表示没有错误，其他任何值表示有错误。
<code>-</code>	显示当前 <code>shell</code> 使用的选项



第9章 Shell编程

❑ 例如：建立shell程序test3，其内容为：

```
#!/bin/bash
```

```
clear
```

```
cal $1 98
```

```
cal $2 98
```

```
cal $3 98
```

❑ 执行时输入命令的形式为：

```
$ ./test3 1 2 3
```

第一个实参值是1，运行时代换给程序中的形参\$1，

第二个实参值是2，运行时代换给程序中的形参\$2，

第三个实参值是3，运行时代换给程序中的形参\$3



第9章 Shell编程

❑ 例：建立shell程序test2，其内容为：

```
#!/bin/bash
```

```
echo $0 $1 $2
```

```
shift
```

```
echo $0 $1 $2
```

左移位置参数

例如：



❑ 运行时情况如下：

```
# /home/test2 yesterday today tomorrow
```

❑ 输出：

```
/root/test2 yesterday today
```

```
/root/test2 today tomorrow
```

❑ 可见：**\$0** 始终是当前运行的shell程序命令本身，即/root/shex2



Shell 变量

□ shell 变量大致可以分为三类：内置变量、用户变量和环境变量。

内置变量：由系统提供，**用户不能修改**（\$@, \$*, \$?, \$0, \$\$, \$!,）。

用户变量：由**用户建立和修改**，在 shell 脚本编写中会经常用到，是在当前shell中使用的**局部变量**，只在当前的Shell环境中有效。

环境变量：这些变量决定了用户工作的环境，它们不需要用户去定义，可以直接在 shell 中使用，其中**某些变量用户可以修改**，**环境变量是全局变量**。



第9章 Shell编程

□ 变量命名

- 变量名必须以字母或下划线开头，只能使用字母、数字或下划线。任何其它字符都标志变量名的结束。
- 变量名大小写敏感。

题目：

□ 下列变量名中，符合Shell命名规范的是？（）

- A. name 1 B. name1 C. \$aa D. 1name



第9章 Shell编程

□ Shell变量命名规则:

1. 必须以字母或下划线开头，长度最长不能超过合法的变量名限制；
2. 一般采用大写字母或下划线的组合，以免和shell内部变量冲突；
3. 注意，变量名不能以数字开头，也不应该使用特殊符号；
4. 变量名可以由字母、数字和下划线组成，注意Linux中的变量名是区分大小写的，例如变量VAR和var是不同的，一般在Linux中采用大写字母来命名；
5. 避免使用shell关键字、shell内置函数名或者扩展名创建变量；
6. 当命名复杂变量时，可以用下划线来划分有意义的词汇，如name_of_variable；
7. 尽量使用有意义的、表意明确的单词命名变量；
8. 尽量少用缩写来命名，比如年份可以命名为year，而不是yr。



第9章 Shell编程

❑ 变量赋值: `variable=value`

- 等号两边不能有空格
- 如果要给变量赋空值，可以在等号后面跟一个换行符

❑ 格式: `<变量名>=<字符串>`

❑ 功能: 将等号右边的字符串赋给等号左边的变量

变量名不能包含空格，因为shell使用空格分析命令、定界命令名和参数，所以若字符串中含有空格，则在字符串上加上引号。

例: `name1=Ubuntu`

`name2="Win 10"`



第9章 Shell编程

❑ 显示变量的值

```
echo $variable 或 echo ${variable}
```

❑ 清除变量

```
unset variable
```

❑ 显示所有变量

```
set
```

例：

```
myname=ubuntu  
echo $myname  
unset myname  
echo $myname
```



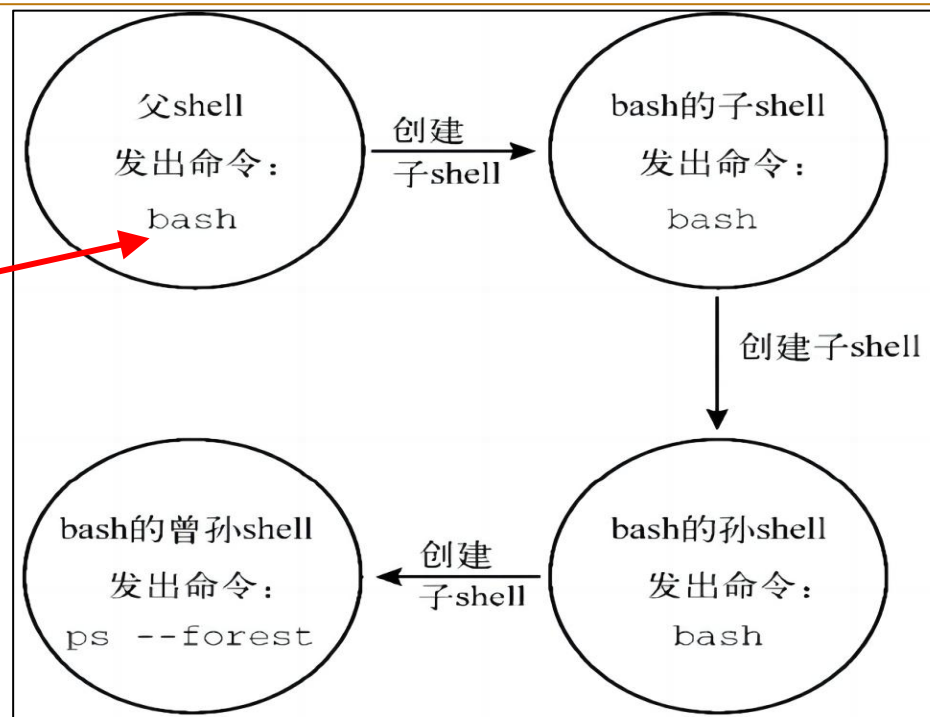
作用域

□ 变量的作用域:

是指变量在一个程序中哪些地方可见。

对于 shell 来说，局部变量的作用域限定在创建它们的 shell 中。

```
[jypan@qtm213 ~]echo $$
5617
[jypan@qtm213 ~]round=world
[jypan@qtm213 ~]echo $round
[jypan@qtm213 ~]bash
[jypan@qtm213 ~]echo $$
5751
[jypan@qtm213 ~]echo $round
[jypan@qtm213 ~]exit
```





引用变量

- ❑ 格式: `$<变量名>`
- ❑ 功能: 引用变量的值, 只要在变量名前加上“\$”字符。
- ❑ 例:

```
name1=zhang
```

```
name2=$name1
```

```
#将变量name1值赋予变量name2
```

```
echo $name2
```

```
#在屏幕上显示变量name2的值
```



第9章 Shell编程

❑ 变量值中有空格:

- 1) 单引号和双引号允许用户一次把几个字符引起来。
- 2) 包含在双引号中有\$字符, 并紧跟变量名, 则在执行shell命令时, 字符串中的变量将被变量所代表的数值所取代。

❑ 例:

```
# winner=zhangsan
```

```
# notice="The person who won is $winner."
```

```
# echo $notice
```

显示结果为: The person who won is zhangsan.



第9章 Shell编程

❑ 换成单引号结果如何？——单引号将抑制任何变量值。

例：

```
# winner=zhangsan
```

```
# notice='The person who won is $winner.'
```

```
# echo $notice
```

显示结果为： ? ? ? ? ?



变量定义和访问

- 单引号：
 - 即使单引号里面有变量和命令也会把它们原样输出。
- 双引号：
 - 先解析里面的变量和命令。
- 反引号：
 - 反引号（`）位于 Esc 键的下方。
 - 反引号主要用于命令替换，等价于 \$()。



第9章 Shell编程

□ 若要将 **\$** 作为普通符号使用，可以加上反斜杠\来引导\$操作符。

例：

```
# winner=zhangsan
```

```
# result="$winner won \"$100.00"
```

```
# echo $result
```

显示结果为： zhangsan won \$100.00.



环境变量

- 1) **作用域**: 包含创建它们的 **shell**, 以及从该 **shell** 产生的任意子 **shell** 或进程。
- 2) 按照惯例, **环境变量通常使用大写**。
- 3) 环境变量是已经用 **export** 内置命令输出的变量。

❑ 变量被创建时所处的 `shell` 称为父 `shell`。如果在父 `shell` 中启动一个新的 `shell` (或进程), 则该 `shell` (或进程) 被称为子`shell` (或子进程)。

❑ 环境变量可以从父亲传递给儿子, 再到孙子, 但不能从子进程传递给父进程。



第9章 Shell编程

- ❑ **export:** 使变量的值对当前shell的所有子进程都可见。
- ❑ 说明: 设置或显示环境变量。
- ❑ 语法: **export** [选项] [变量名称]=[变量设置值]

该命令的各选项含义如下:

- f** 代表[变量名称]中为函数名称;
- n** 删除指定变量, 变量实际未删除, 只是不输出到后续指令的执行环境中;
- p** 列出所有shell赋予程序的环境变量。



第9章 Shell编程

□ 设置环境变量

```
export variable=value
```

```
variable=value; export variable
```

例： | [jypan@fish213 ~]\$ echo \$\$

```
[jypan@fish213 ~]$ export round=world
[jypan@fish213 ~]$ bash
[jypan@fish213 ~]$ echo $$
15175
[jypan@fish213 ~]$ echo $round
```

```
export -n variable
```

将全局变量转换成局部变量

```
export -p
```

列出所有全局变量



常见的系统环境变量

变量名	含义
HOME	用户主目录
UID	当前用户的标识符，取值是由数字构成的字符串
USER	用户名，与登录名相同
PWD	当前工作目录的绝对路径名，该变量的取值随cd命令的使用而变化
MAIL	用户的邮箱路径名
HOSTNAME	计算机的主机名
INPUTRC	默认的键盘映像
SHELL	用户所使用的 shell 的路径名
HISTSIZE	history 所能记住的命令的最多个数



第9章 Shell编程

变量名	含义
PATH	shell 查找用户输入命令的路径（目录列表），shell将按PATH变量中给出的顺序搜索这些目录，找到的第一个与命令名称一致的可执行文件将被执行
PS1	shell一级命令提示符（主提示符） 在特权用户下，默认的主提示符是#；在普通用户下，默认的主提示符是\$
PS2	shell 二级命令提示符 在shell接收用户输入命令的过程中，如果用户在输入行的末尾输入“\”然后按回车键，或者当用户按回车键shell判断出用户输入的命令没有结束时，就显示这个辅助提示符，默认是“>”



第9章 Shell 编程

❑ **PATH 变量**是最重要的环境变量之一。当用户在命令行中输入命令时，**shell 就会根据该变量定义的路径（目录）和顺序，查找并执行该命令**。如果没有正确设置 PATH 变量，则必须输入完整的路径名来运行某个命令。

❑ 在 Linux 下输入命令的两种方式：

- ✓ 直接在命令行中输入命令：根据 PATH 查找该命令
- ✓ 输入完整的路径名

❑ 用户可以根据需要修改环境变量
如： **# PATH=\$PATH:/home/scripts**



第9章 Shell编程

❑ IFS 变量是一个环境变量，用于指定字段之间的分隔符。

❑ 在 Linux 下输入命令的两种方式：

- ✓ 直接在命令行中输入命令：根据 PATH 查找该命令
- ✓ 输入完整的路径名

❑ 用户可以根据需要修改环境变量
如： **# PATH=\$PATH:/home/scripts**



变量值输出

- 1. echo输出
 - -e: 识别输出内容里面的转义字符;
 - -n: 忽略结尾的换行。
- 2. printf输出
 - 换行需要自己加 “\n”

```
zp@lab:~/shell$ echo "Hello\tShell!"  
Hello\tShell!  
zp@lab:~/shell$ echo -e "Hello\tShell!"  
Hello  Shell!  
zp@lab:~/shell$ echo -n "Hello\tShell!"  
Hello\tShell!zp@lab:~/shell$
```

```
zp@lab:~/shell$ printf "Hello\tShell"  
Hello  Shellzp@lab:~/shell$  
zp@lab:~/shell$ printf "Hello\tShell\n"  
Hello  Shell
```



第9章 Shell编程

- printf 可用来按指定的格式输出变量

printf format 输出参数列表

printf 的打印格式与 C 语言中的 printf 相同

```
printf "%-12.5f\n" 123.456
```

format
以%开头

格式符

flag

field width

precision

-: 左对齐
+: 输出符号
0: 空白处添0
空格: 前面加一空格

字段宽度

小数点后
输出位数



第9章 Shell编程

printf 命令的格式说明符

c	字符型	g/G	浮点数（自动）
d	十进制整数	o	八进制
e/E	浮点数（科学计数法）	s	字符串
f	浮点数（小数形式）	x/X	十六进制

format 中还可以使用

\a	警铃	\t	水平制表符
\b	退后一格	\v	垂直制表符
\n	换行	\\	反斜杠
\f	换页	\”	双引号
\r	回车	%%	百分号



第9章 Shell编程

例:

```
printf "The number is: %.2f\n" 100
```

```
printf "%-20s|%12.5f|\n" "Joy" 10
```

```
printf "%-10d%010o%+10x\n" 20 20 20
```

```
printf "%6d\t%6o\ \"%6x\ "\n" 20 20 20
```



变量输入（赋值）：

- `read [参数] [变量名]`
 - `read -s -p "Enter Password:" password`
 - `echo -e "\nThe password your input is:$password"`

```
zp@lab:~/shell$ read -s -p "Enter Password:" password
Enter Password:zp@lab:~/shell$
zp@lab:~/shell$
zp@lab:~/shell$ echo -e "\nThe password your input is:$password"

The password your input is:111111
```

- `read -n 8 -t 10 -s -p "Enter Password:" password`

- `read first last`
- `echo $first`
- `echo $last`

```
zp@lab:~/shell$ read first last
zhang ping
zp@lab:~/shell$ echo $first
zhang
zp@lab:~/shell$ echo $last
ping
```



第9章 Shell编程

编写一个Shell脚本，并根据以下要求进行编写：

- ❑ 在脚本的开头，使用注释来解释脚本的用途和编写者信息
- ❑ 定义一个环境变量DATA_HOME，该变量表示数据文件所在的目录，给该变量赋值为/home/data
- ❑ 定义一个脚本形参input_file，表示用户指定的输入文件
- ❑ 定义一个脚本形参output_file，表示用户指定的输出文件
- ❑ 使用cat命令读取input_file文件的内容，并使用sed命令将其中的所有空格替换为“_”（下划线）字符。（提示： `cat "$input_file" | sed 's/ /_/g'`）
- ❑ 将替换后的文本输出到output_file中
- ❑ 在脚本末尾，使用 `ls -al` 命令列出DATA_HOME目录下所有的文件和文件夹，以检查是否生成了新的输出文件



第9章 Shell编程



第9章 Shell编程

- \$0 当前脚本名
- \$* 传递给脚本的所有参数
- \$\$ 当前脚本的pid
- \$! 后台运行最后一个进程的pid
- \$? 上一个命令是否执行成功，成功为0，否则为1
- \$#脚本的参数个数



Shell 支持两种分支结构（选择结构）

if else 语句

```
if expr1      # 如果expr1 为真(返回值为0)
then          # 那么
    commands1 # 执行语句块 commands1
elif expr2    # 若expr1 不真, 而expr2 为真
then          # 那么
    commands2 # 执行语句块 commands2
... ..       # 可以有多个 elif 语句
else          # else 最多只能有一个
    commands4 # 执行语句块 commands4
fi            # if 语句必须以单词 fi 终止
```

case in 语句

```
case expr in  # expr 为表达式, 关键词 in 不要忘!
    pattern1) # 若 expr 与 pattern1 匹配, 注意括号
        commands1
        ;;    # 跳出 case 结构
    pattern2) # 若 expr 与 pattern2 匹配
        commands2
        ;;    # 跳出 case 结构
    ... ..    # 可以有任意多个模式匹配
    *)        # 若 expr 与上面的模式都不匹配
        commands
        ;;    # 跳出 case 结构
esac          # case 语句必须以 esac 终止
```



□ 语法结构

重点及难点

```
if expr1      # 如果expr1 为真(返回值为0)
then          # 那么
    commands1 # 执行语句块 commands1
elif expr2    # 若expr1 不真, 而expr2 为真
then          # 那么
    commands2 # 执行语句块 commands2
... ..       # 可以有多个 elif 语句
else          # else 最多只能有一个
    commands4 # 执行语句块 commands4
fi            # if 语句必须以单词 fi 终止
```

- 1) if 开头, fi 结尾
- 2) if 如果 和 then 在一行, 需要加 ;



第9章 Shell编程

例：

给定一个用户，来判断这个用户是什么用户，如果是管理员用户，则显示“该用户为管理员”，否则显示“该用户为普通用户”

1、whoami（显示当前用户的用户名）

```
if [ `whoami` = "root" ]; then
    echo "该用户为管理员用户"
else
    echo "该用户为普通用户"
fi
```

2、id -u（显示当前用户的uid）

```
if [ `id -u` -eq 0 ]; then
    echo "该用户为管理员用户"
else
    echo "该用户为普通用户"
fi
```



条件判断：

- 条件判断可以根据某个特定条件是否满足，来选择执行相应的任务。

- Bash 中允许判断两种类型的条件：

命令成功或失败，表达式为真或假

- 任何一种测试中，都要有退出状态（返回值），退出状态为0表示命令成功或表达式为真，非0则表示命令失败或表达式为假。



❑ 表达式测试包括字符串比较、整数比较和文件测试。

条件判断格式

- 格式1: 用 `test` 命令来测试表达式的值

```
x=5; y=10
test $x -gt $y
echo $?
```

- 格式2: 用方括号来测试表达式的值，每一项两边都要有空格

```
x=5; y=10
[ $x -gt $y ]
echo $?
```

方括号前后要留空格！



□ 1) 字符串比较

操作符两边必须留空格！

[-z str]	如果字符串 str 长度为 0，返回真
[-n str]	如果字符串 str 长度不为 0，返回真
[str1 = str2]	两字符串相等
[str1 != str2]	两字符串不等

例1: name=Tom; [-z \$name]; echo \$?

例2: name2=Andy; [\$name = \$name2] ; echo \$?



第9章 Shell编程

例：

从标准输入读入输入的密码，并判断密码是否正确：

```
read -sp "please enter your password: " passwd #输入的密码不在屏幕上显示
if [ "$passwd" = "test123" ]
then
    echo -e "\n password verified"
    exit 0      #如果变量pass的值为test123，则显示密码为通过验证的信息，
                #然后退出脚本的执行，退出的状态码为0
fi
echo -e "\n your input is wrong"      #开启转义
```



□ 2) 整数比较, 即比较大小

操作符两边必须留空格!

[int1 -eq int2]	int1 等于 int2
[int1 -ne int2]	int1 不等于 int2
[int1 -gt int2]	int1 大于 int2
[int1 -ge int2]	int1 大于或等于 int2
[int1 -lt int2]	int1 小于 int2
[int1 -le int2]	int1 小于或等于 int2

例: x=1; [\$x -eq 1]; echo \$?

例: x=a; [\$x -eq 1]; echo \$?

X



例：

编写一个猜年龄的游戏程序，程序设置一个默认年龄，然后让用户猜年龄，如果刚好跟你设置的年龄一致，则提示用户“你猜对了”，否则提示“你猜错了”。（这个脚本需要判断用户输入的是否是数字，如果输入的不是，需要提示用户输入的错误。）



第9章 Shell编程

```
#!/bin/bash
myage=20
read -t 10 -p "请输入年龄: " age
if [ $? -eq 0 ]
then
    if [ $age -gt $myage ]
    then
        echo "你猜大了!"
    elif [ $age -lt $myage ]
    then
        echo "你猜小了"
    else
        echo "你猜对了!"
    else
        echo "你输入的不是数字!"
    fi
fi
```



❑ 整数比较也可以使用 `let` 命令或双圆括号

- 相应的操作符为：只能用于整数比较！

`==` 、 `!=` 、 `>` 、 `>=` 、 `<` 、 `<=`

- 例：

```
x=1; let "$x == 1"; echo $?
```

```
x=1; (( $x+1 >= 2 )); echo $?
```

❑ 两种比较方法的区别

- 使用的操作符不同；
- `let` 和 双圆括号中可以使用算术表达式，而中括号不能；
- `let` 和 双圆括号中，操作符两边可以不留空格。



第9章 Shell编程

比较浮点数:

浮点数的比较有点特殊，无法使用字符串和整型的比较方式，需要使用到额外的命令，命令有好几种，选择内置的命令**expr**，以免去安装

```
#!/bin/bash
a=1.1
b=2.2
c=1.1
d=1.4
if [[ `expr $a == $c` == 1 ]];then
    echo "a == c"
fi
if [[ `expr $a != $b` == 1 ]];then
    echo "a != b"
fi
```

```
if [[ `expr $b \> $a` == 1 ]];then
    echo "b > a"
fi
if [[ `expr $c \>= $a` == 1 ]];then
    echo "c >= a"
fi
if [[ `expr $a \< $b` == 1 ]];then
    echo "a < b"
fi
if [[ `expr $c \<= $a` == 1 ]];then
    echo "c <= a"
fi
```



第9章 Shell编程



□ 3) 文件测试：文件是否存在，文件属性，访问权限等。

常见的文件测试操作符：

-f fname	fname 存在且是普通文件时，返回真（即返回 0）
-L fname	fname 存在且是链接文件时，返回真
-d fname	fname 存在且是一个目录时，返回真
-e fname	fname（文件或目录）存在时，返回真
-s fname	fname 存在且大小大于 0 时，返回真
-r fname	fname（文件或目录）存在且可读时，返回真
-w fname	fname（文件或目录）存在且可写时，返回真
-x fname	fname（文件或目录）存在且可执行时，返回真



第9章 Shell编程

例： 判断目录是否存在，不存在则创建。

```
#!/bin/bash
#date: 2022-4-13
#描述: if单分支语句示例，判断目录是否存在
read -p "请输入需要判断的目录: " name
if [ ! -d $name ]
then
    echo "$name目录不存在，正在创建..."
    mkdir -p $name
    echo "$name目录创建完成."
fi
echo "$name目录已存在，正在退出..."
```



第9章 Shell编程

例：

如果/etc目录中的文件profile存在，则将其复制到/home中，并在屏幕上显示/home下的profile内容；否则在屏幕上显示信息“profile is not exist!”

```
#!/bin/bash
if test -e /etc/profile ; then
    cp /etc/profile /home
    cat /etc/profile | more
else
    echo “profile is not exist!”
fi
```




第9章 Shell编程

例：创建一个脚本，脚本作用如下：

1).先查看/tmp/test这个文件是否存在；

2).若不存在则创建一个文件，建立完后退出；

3).若存在则判断该名称是否是文件，若为文件，则将其删除后创建一个同名目录，之后退出；

4).如果存在且为目录则删除此目录。

```
#!/bin/bash
if [ -e "/tmp/test" ]           #判断文件是否存在
then                             #文件存在时执行下面语句
    if [ -f "/tmp/test" ]       #判断是否普通文件
    then
        rm -rf /tmp/test
        mkdir -p /tmp/test
    elif [ -d "/tmp/test" ]      #判断是否为目录
    then
        rm -rf /tmp/test
    fi
else
    touch /tmp/test              #不存在创建
    exit 0
fi
```



第9章 Shell编程

例：

判断用户输入的是文件还是目录。

```
#!/bin/bash
#date:2022-4-13
#描述:判断文件类型
read -p "请输入一个文件:" file
if [ -z $file ]; then
    echo "错误！输入的文件为空."
elif [ !-e $file ]; then
    echo "错误！输入的文件不存在."
elif [ -f $file ]; then
    echo "$file是一个普通文件"
elif [ -d $file ]; then
    echo "$file是一个目录"
else
    echo "$file是其他类型文件"
fi
```



□ 检查空值

```
[ "$name" = "" ]
```

```
[ ! "$name" ]
```

```
[ "X${name}" != "X" ]
```



□ 逻辑符号

[expr1 -a expr2]	逻辑与，都为真时，结果为真
[expr1 -o expr2]	逻辑或，有一个为真时，结果为真
[! expr]	逻辑非

```
x=1; name=Tom;
```

```
[ $x -eq 1 -a -n $name ]; echo $?
```

注：不能随便添加括号

```
[ ( $x -eq 1 ) -a ( -n $name ) ]; echo $?
```

X



□ 可以使用模式的逻辑测试

<code>[[pattern1 && pattern2]]</code>	逻辑与
<code>[[pattern1 pattern2]]</code>	逻辑或
<code>[[! pattern]]</code>	逻辑非

```
x=1; name=Tom;
```

```
[[ $x -eq 1 && $name = To? ]]; echo $?
```

- 使用`[[...]]`条件判断结构，而不是`[...]`，能够防止脚本中的许多逻辑错误。
- 比如，`&&`、`||`、`<`和`>`操作符能够正常存在于`[[...]]`条件判断结构中，但是如果出现在`[...]`结构中的话，会报错。
- 比如可以直接使用`if [[ $a -ne 1 && $a -ne 2 ]]`，如果不使用双括号，则为`if [ $a -ne 1 ] && [ $a -ne 2 ]`或者`if [ $a -ne 1 -a $a -ne 2 ]`。



第9章 Shell编程

- 1) `command1 ; command2`
- 2) `command1 && command2`
- 3) `command1 || command2`

1—使用“**;**”时，不管**command1**是否执行成功都会执行**command2**;

2—使用“**&&**”时，只有**command1**执行成功后，**command2**才会执行，否则**command2**不执行;

3—使用“**|**”时，**command1**执行成功后**command2**不执行，否则去执行**command2**，总之**command1**和**command2**总有一条命令会执行。



几点说明：

- ◆ `elif` 可以有任意多个（0 个或多个）
- ◆ `else` 最多只能有一个（0 个或 1 个），`else`后面不用加`then`
- ◆ `if` 语句必须以 `fi` 表示结束
- ◆ `expr` 通常为条件测试表达式；也可以是多个命令，以最后一个命令的退出状态为条件值。
- ◆ `commands` 为可执行语句块，如果为空，需使用 `shell` 提供的空命令“`:`”，即冒号。该命令不做任何事情，只返回一个退出状态 0
- ◆ `if` 语句可以嵌套使用



第9章 Shell编程

例：

建立脚本文件shp3，存放在当前目录下。如果shp3运行时未带参数，则在屏幕上显示信息“Parameter is lost!”，并结束程序运行；如果shp3运行时带一个参数，则判断参数所指定的文件是否存在，如果存在则复制该文件到/home；否则先在屏幕上显示信息“File not found!”，然后显示程序名称本身。



第9章 Shell编程

例：

建立脚本文件test1.sh，存放在当前目录下。

1) 如果test1.sh运行时未带参数，则在屏幕上显示信息“Parameter is lost!”，并结束程序运行；

2) 如果test1.sh运行时带一个参数，则判断参数所指定的文件是否存在，如果存在则复制该文件到/home；

3) 否则先在屏幕上显示信息“File not found!”，然后显示程序本身。

```
#!/bin/bash
if [ -z $1 ]                #-z判$1存在否
then echo Parameter is lost! #如带一参数
elif [ -e $1 ]             #则$1肯定在
then cp $1 /home
else
echo File not found!
cat $0                      #$0指程序本身
fi
```



第9章 Shell编程

例：

- 1) 编写shell程序test2.sh，存放在当前目录下。运行test2.sh时带一个正整数参数，将数值存入变量value；
- 2) 如果该参数大于等于1但小于3，则在屏幕上显示“value is less than 3 and not less than 1”；
- 3) 如果该参数大于等于3，则在屏幕上显示“value is not less than 3”；
- 4) 最后以“value=xx”的形式显示变量value的值。

```
#!/bin/bash
```

```
value=$1
```

```
if [ $value -ge 1 -a $value -lt 3 ]
```

```
then echo "value is less than 3 and not less than 1"
```

```
elif [ $value -ge 3 ]
```

```
then echo "value is not less than 3"
```

```
fi
```

```
echo "value=$value"
```



第9章 Shell编程

假设你是一家图书馆的管理员，图书馆有一个会员系统，每个会员都有一个唯一的会员号。图书馆会生成一个包含借阅记录的文本文件**borrow_records.txt**，每行记录格式为：会员号,书籍名称,借阅日期。

borrow_records.txt文件内容示例：

```
12345,BookA,2023-04-01
67890,BookB,2023-04-02
12345,BookC,2023-04-03
34567,BookD,2023-04-04
12345,BookB,2023-04-05
```

现在，你需要编写一个shell脚本，实现以下功能：

- 1) 读取**borrow_records.txt**文件，检查是否存在某个特定会员（比如会员号为**12345**）的借阅记录。
- 2) 如果存在该会员的借阅记录，输出其借阅的所有记录。
- 3) 如果该会员借阅的记录数量超过5条，则发送一封电子邮件通知管理员。

- ①使用**grep "\$target_member" borrow_records.txt**命令筛选出目标会员的借阅记录。
- ②使用**-n**选项检查变量**borrowed_books**是否为空，如果不为空，说明找到了借阅记录。
- ③使用**echo**输出该借阅记录。
- ④使用**wc -l**计算借阅的记录数量。
- ⑤使用**if**语句判断记录数量是否超过5条，如果是，则使用**mail -s "借阅提醒" admin@library.com**命令发送邮件通知管理员。



第9章 Shell编程



□ 语法结构

```
case expr in
    pattern1)
        commands1
        ;;
    pattern2)
        commands2
        ;;
    ... ..
    *)
        commands
        ;;
esac
```

expr 为表达式, 关键词 in 不要忘记!

若 expr 与 pattern1 匹配, 注意括号

执行语句块 commands1

跳出 case 结构

若 expr 与 pattern2 匹配

执行语句块 commands2

跳出 case 结构

可以有任意多个模式匹配

若 expr 与上面的模式都不匹配

执行语句块 commands

跳出 case 结构

case 语句必须以 esac 终止

```
#!/bin/bash
printf "Input integer number: "
read num
case $num in
    1)
        echo "Monday"
        ;;
    2)
        echo "Tuesday"
        ;;
    3)
        echo "Wednesday"
        ;;
    4)
        echo "Thursday"
        ;;
    5)
        echo "Friday"
        ;;
    6)
        echo "Saturday"
        ;;
    7)
        echo "Sunday"
        ;;
    *)
        echo "error"
esac
```



第9章 Shell编程

几点说明:

- ❑ expression 既可以是一个变量、一个数字、一个字符串，还可以是一个数学计算表达式，或者是命令的执行结果，只要能够得到 expression 的值就可以。
- ❑ pattern 可以是一个数字、一个字符串，甚至是一个简单的正则表达式。
- ❑ 可以没有*)部分。如果 expression 没有匹配到任何一个模式，那么就不执行任何操作。
- ❑ 除最后一个分支外（这个分支可以是普通分支，也可以是*)分支），其它的每个分支都必须以;;结尾。
- ❑ 如果不加以说明，Shell 的值都是字符串，expression 和 pattern 也是按照字符串的方式来匹配的。



第9章 Shell编程

case in 的 pattern 部分支持简单的正则表达式

格式	说明
*	表示任意字符串。
[abc]	表示 a、b、c 三个字符中的任意一个。比如，[15ZH] 表示 1、5、Z、H 四个字符中的任意一个。
[m-n]	表示从 m 到 n 的任意一个字符。比如，[0-9] 表示任意一个数字，[0-9a-zA-Z] 表示字母或数字。
	表示多重选择，类似逻辑运算中的或运算。比如，abc xyz 表示匹配字符串“abc”或者“xyz”

例：字符识别输入数字or字母 ➡

```
#!/bin/bash
printf "Input a character: "
read -n 1 char
case $char in
    [a-zA-Z])
        printf "\nletter\n"
        ;;
    [0-9])
        printf "\nDigit\n"
        ;;
    [.,?!])
        printf "\nPunctuation\n"
        ;;
    *)
        printf "\nerror\n"
esac
```



第9章 Shell编程

例：

判断今天是星期几，根据星期几来判断 今天是工作日还是休息日。

```
#!/bin/bash
a=`date +%w` #将日期转换为星期几，用数字0-6表示
case $a in
[1-5])
    echo "今天是工作日，请认真工作" ;;
[06])
    echo "哦~今天是休息日呢，请好好休息" ;;
esac
```




第9章 Shell编程

例：根据输入判断程序是否执行

```
#!/bin/bash
read -p "Do you agree(yes/no)? " INPUT
case $INPUT in
[yY]|[Yy][Ee][Ss])
    echo "You input is YES"
    ;;
[Nn]|[Nn][Oo])
    echo "You input is NO"
    ;;
*)
    echo "Input fales, please input yes or no!"
esac
```



几点说明：

- ◆ 表达式 `expr` 按顺序匹配每个模式，一旦有一个模式匹配成功，则执行该模式后面的所有命令，然后退出 `case`。
- ◆ 如果 `expr` 没有找到匹配的模式，则执行缺省值 “`*)`” 后面的命令块（类似于 `if` 中的 `else`）；“`*)`” 可以不出现。
- ◆ 所给的匹配模式 `pattern` 中可以含有通配符和 “`|`”。
- ◆ 每个命令块的最后必须有一个双分号，可以独占一行，或放在最后一个命令的后面。



第9章 Shell编程

例：成绩判断

```
#!/bin/bash
#Determine grades and scores

read -p "请输入你的分数（0-100）：" score

[ $score -ge 90 -a $score -le 100 ] && a="great"
[ $score -ge 70 -a $score -le 89 ] && a="medium"
[ $score -ge 60 -a $score -le 69 ] && a="pass"
[ $score -lt 60 ] && a="fail"
case $a in
great)
    echo "优秀"
;;
```

```
medium)
    echo "中等"
;;
pass)
    echo "及格"
;;
fail)
    echo "不及格"
;;
*)
    echo "输入有误"
esac
```



第9章 Shell编程



第9章 Shell编程



□ 语法结构

重点及难点

```
for variable in list      # list——取值列表
    # 每一次循环，依次把列表 list 中的一个值赋给循环变量
do                          # 循环开始的标志
    commands               # 循环变量每取一次值，循环体就执行一遍
done                       # 循环结束的标志
```

□ 几点说明

- 列表 `list` 可以是命令替换、变量名替换、字符串和文件名列表（可包含通配符）
- `for` 循环执行的次数取决于列表 `list` 中单词的个数
- `for` 循环体中一般要出现循环变量，但也可以不出现



第9章 Shell编程

- 列表 `list` 可以是命令替换、变量名替换、字符串和文件名列表（可包含通配符）

```
#!/bin/bash
for test in I "don't" know
do
    echo "word:$test"
done

$ ./test2
```

列表

```
#!/bin/bash
list="one two three four"
list=$list" five"
for test in $list
do
    echo "word:$test"
done

$ ./test2
```

变量

```
#!/bin/bash
file="states.txt"

for test in $(cat $file)
do
    echo "word:$test"
done

$ ./test2
```

命令



第9章 Shell编程

- 更改字段分隔符——环境变量：IFS

bash shell在数据中看到这些字符中的任意一个，它就认为这是列表中的一个新字段的开始。

默认情况下，字段分隔符为：

- ☐ 空格
- ☐ 制表符（Tab）
- ☐ 换行符

例如，如果想利用for循环获取文件内容，该如何编写脚本？



第9章 Shell编程

- 更改字段分隔符——环境变量：IFS

例如，如果想利用for循环获取文件内容，该如何编写脚本？

```
#!/bin/bash
#changing the IFS value

IFS.OLD=$IFS
IFS=$'\n'
for entry in $(cat /etc/passwd)
do
    echo "Value in $entry -"
    IFS=:
    for value in $entry
    do
        echo "    $value"
    done
done
```



第9章 Shell编程

例：遍历列表

```
#!/bin/bash

for val in 1 2 3 4 5
do
    echo -n "$val"
done
```

例：累加求和

```
#!/bin/bash
sum=0
for i in {1..100}
do
    let sum=$i+$sum
    sum=$((i+sum))
done
echo sum=$sum
```



第9章 Shell编程

例：

用for命令实现，在当前目录下创建名为user0、user1、...user9十个子目录，用长格式显示这10个目录的目录信息，然后用for命令删去这10个目录。

```
for i in 0 1 2 3 4 5 6 7 8 9
do
    mkdir user$i
done
ls -dl user?|more
for i in 0 1 2 3 4 5 6 7 8 9
do
    rm -rf user$i
done
```



第9章 Shell编程

例：

检查/home目录下，是否存在由参数1指定的文件：

1) 若不存在，则屏幕显示信息“File not found!”；

2) 若存在，则将参数1指定的文件改名为由参数2指定的文件名，然后用for命令对改名后的文件显示其长格式的目录信息和文件内容。



第9章 Shell编程

```
if test -n $1 -a -n $2; then
    if test $1 != $2; then
        if test -e $1; then
            mv $1 $2
            for cn in "ls -l" "cat"
            do
                $cn $2
                echo " "
            done
        else echo File not found!
        fi
    fi
fi
```



□ 结构实例

```
for (( i=1;i<=5;i++))  
do  
    echo $i  
done
```

□ 几点说明

- 1.变量赋值可以有空格;
- 2.迭代条件中的变量不以美元符号开头;
- 3.迭代过程的算式不用**expr**命令格式



第9章 Shell编程

●作业:

用for循环算出 某山顶上有一颗香蕉树，一只猴子第一天从树上摘了若干根香蕉，当即就吃了一半，还不过瘾，又多吃了一根。第二天猴子又将剩下的香蕉吃了一半，禁不住诱惑，又多吃了一根香蕉。依此类推，每天都将剩余的香蕉吃一半后再多吃一根。到了第十天，猴子发现只剩一根香蕉了，请问这只猴子在第一天总共摘了多少根香蕉？



□ 语法结构

```
while expr          # 执行 expr
do                  # 若 expr 的退出状态为0，进入循环，否则退出while
    commands        # 循环体
done                # 循环结束标志，返回循环顶部
```

□ 执行过程

先执行 `expr`，如果其退出状态为 `0`，就执行循环体。执行到关键字 `done` 后，回到循环的顶部，`while` 命令再次检查 `expr` 的退出状态。以此类推，循环将一直继续下去，直到 `expr` 的退出状态非 `0` 为止。



第9章 Shell编程

```
#!/bin/bash
```

```
i=0
```

```
while [ $i -lt 10 ]
```

```
do
```

```
    let i++
```

```
    echo $i
```

```
done
```



循环控制 shift 命令

`shift [n]`

- 用于将参量列表 `list` 左移指定次数，缺省为左移一次。
 - 参量列表 `list` 一旦被移动，最左端的那个参数就从列表中删除。
- `while` 循环遍历位置参量列表时，常用到 `shift`。

例：

```
test.sh a b c d e f g h
```

```
shift.sh a b c d e f g h
```



第9章 Shell编程

假设你是一名管理员，你需要编写一个Shell程序来管理Linux系统上的用户账户。请编写一个Shell程序实现以下功能：

- ❑ 提示用户选择需要进行的操作：**添加用户**、**删除用户**、**修改密码**、**查看用户列表**。
- ❑ 根据用户输入的操作，进行相应的处理：
 - ❑ 添加用户：接收用户输入的用户名和密码，并添加到系统中。
 - ❑ 删除用户：接收用户输入的用户名，并删除该用户。
 - ❑ 修改密码：接收用户输入的用户名和新密码，并修改该用户的密码。
 - ❑ 查看用户列表：列出系统中的所有用户。
- ❑ 在每次操作后，提示用户是否继续操作。如果用户输入"Y"，则重新提示用户输入需要进行的操作；如果用户输入"N"，则退出程序。如果用户输入其他内容，则重新提示用户输入。

提示：在这个Shell程序中，使用一个无限循环来提示用户输入需要进行的操作。使用read命令读取用户的选择，并使用case语句进行分类处理。

对于添加用户操作，使用useradd命令添加新用户，使用chpasswd命令将密码保存。对于删除用户操作，使用userdel命令删除用户。对于修改密码操作，使用**echo "\$username:\$password" | sudo chpasswd**命令修改密码。对于查看用户列表操作，使用cat命令从/etc/passwd文件中提取用户列表。

在每次操作后，询问用户是否继续操作，如果用户输入"Y"，则重新提示用户输入需要进行的操作；如果用户输入"N"，则退出程序。如果用户输入其他内容，则重新提示用户输入。



第9章 Shell编程



□ ()

● 1、命令组——()

连接多个命令组，用 `;` 隔开，最后一个命令后 `;` 可加可不加。括号中的命令会新开一个子shell顺序执行，所以里面的操作对括号外主进程无关。

```
a=0
(a=1;b=2;echo $a) # 子进程内输出a为1
echo $a # 主进程，输出a为0
```

● 2、命令替换——\$()

格式为 `$(command)`，相当于 ``command``。shell遇到此格式，会先执行 `command` 命令，得到标准输出，再将此输出放回到原来命令。

● 3、初始化数组——()

Shell 数组用括号来表示，元素用“空格”符号分割开。

```
array=(a b c)
echo ${array[1]} # 输出b
```



□ (())

● 1、C语言规则运算——\$(())

表达式 `$(exp)`，其中exp只要符合C语言规则的运算符即可，包括加减乘除、+=、<=、>=等。

```
echo $((2*2+(1-2))) # 输出3
a=1
echo $((a++)) # 输出2，且从此之后a=2
echo $((a<2)) # 输出1。如果为假，则输出0
```

● 2、重定义变量值——(())

在`((exp))`中可以对变量进行定义或重新赋值，且之后脚本全部有效（不是像`()`那样只在子进程有效）。

```
a=1
((a=2))
echo $a # 输出2
```

● 3、算术比较——(())

在`((exp))`中可以进行算术比较（不能进行字符串比较），双括号中的变量可以省略\$符号前缀，也可以带着。

```
a=1
((a==1)) && echo "true" # 输出 true
(($a==1)) && echo "true" # 输出 true
```



□ []

● 1、条件判断——[]

[] 本质上是 test 语句，[是调用test的命令标识，]是关闭条件判断，如：

```
a="abc"
```

```
if [ -n "$a" ]; then echo "true"; fi # 输出true，注意[]内-n或者-z时字符串两边必须加上双引号
```

```
if test -n "$a"; then echo "true"; fi # 输出true，注意test内-n或者-z时字符串两边必须加上双引号
```

● 2、字符范围——正则表达式

用作正则表达式的一部分，描述一个匹配的字符范围，如删除一个字符串中的所有数字：

```
echo "ap1p23le" | tr -d "[0-9]" # 输出 apple
```

```
echo [a-b].txt # 输出:a.txt b.txt，前提是两个文件都存在
```

```
# 但是如果当前路径只存在a.txt或b.txt，则只会输出存在的文件
```

```
# 如果两个文件均不存在，则输出固定字符串:[a-b].txt
```

● 3、数组下标

在一个array后，作为引用数组元素的下标，如

```
a=(1 2 3)
```

```
echo ${a[0]} # 输出:1
```

```
echo ${a[*]} # 输出:1 2 3
```



□ [[]]

● 1、条件判断

□ &&、||、< 和 > 操作符能够正常存在于[[]]条件判断结构中，但是如果出现在[]结构中的话，会报错。比如可以直接使用if [[\$a != 1 && \$a != 2]], 如果不使用双括号，则为if [\$a != 1] && [\$a != 2]或者if [\$a != 1 -a \$a != 2]

□ [[]]支持字符串的模式匹配，[]不支持。比如[["hello" == hell?]]为真。模式匹配不能加引号，否则会作为固定字符串，如[["hello" == "hell?"]]为假。

□ [[]]支持算术扩展，而[]不支持。如if [[1+2 -eq 3]], 而if [1+2 -eq 3]则会报错。

● 2、返回状态码

可以看到，如果[[]]内的表达式为真，则返回退出码0，这也是其他所有命令执行正常时所返回的退出码。

```
[[ 1 -gt 0 ]]  
echo $? # 输出0  
[[ 1 -gt 2 ]]  
echo $? # 输出1
```




{ }

- 1、字符扩展 { }可以作为通配符进行扩展，有两种用法：

- ❑ `echo {a,b}.txt` 输出a.txt b.txt。与`echo [a-b].txt`区别在于，后者只输出存在的文件名，而前者不管是不是存在此文件，永远都会输出a.txt b.txt，因为{}的字符扩展和是不是文件名没有关系。

- ❑ `echo {a..c}.txt`输出a.txt b.txt c.txt。

```
$ for i in {0,2};do echo $i;done
0
2
$ for i in {0..2};do echo $i;done
0
1
2
```

- 2、代码块
- 3、特殊替换结构



第9章 Shell编程



shell函数定义

- 函数function是由若干条shell命令组成的语句块，实现代码重用和模块化编程，它与shell程序形式上是相似的，不同的是它不是一个单独的进程，不能独立运行，而是shell程序的一部分。
- 格式：

```
1 function 函数名 {  
2     命令序列  
3 }
```

```
1 函数名 () {  
2     命令序列  
3 }
```

```
1 #!/bin/bash  
2 demoFun(){  
3     echo "这是我的第一个 shell 函数!"  
4 }  
5 echo "-----函数开始执行-----"  
6 demoFun  
7 echo "-----函数执行完毕-----"  
8
```



函数返回值

- `return`表示退出函数并返回一个退出值，脚本中可以用 `$?` 变量显示该值

使用原则：

- 1) 退出状态码可以由 `return` 命令明确指定，也可以由函数中最后一条命令的退出状态码来指定（如果成功，则返回0，否则返回非0 值）。
- 2) 退出状态码必须是 $0 \sim 255$ ，超出时值将为除以 256 取余



第9章 Shell编程

```
$ vim test.sh
```

```
-----  
#!/bin/bash
```

```
function test01 {  
    read -p "请输入一个整数: " num  
    return ${num * 2}  
}
```

```
-----  
$ test01
```

```
$ echo $?
```



函数变量的作用范围

- 脚本中的变量默认全局有效(即函数体内外都有效)

```
$ vim test1.sh
```

```
-----  
#!/bin/bash
```

```
myfun() {  
    echo $name  
    name=lisi  
    echo $name  
    name=zhangsan  
}
```

```
-----  
$ myfun  
$ echo $name
```

- local定义的变量值只能在函数体内部使用，出了函数体将无法使用；

```
$ vim test2.sh
```

```
-----  
#!/bin/bash
```

```
myfun() {  
    echo $name  
    local name=lisi  
    echo $name  
}
```

```
-----  
$ name=zhangsan  
$ myfun  
$ echo $name
```



第9章 Shell 编程

传参:

- 在函数体内部，通过 `$n` 的形式来获取参数的值，例如，`$1`表示第一个参数，`$2`表示第二个参数...

1.第一种方法

```
sum1() {  
    sum=${$1 + $2}  
    echo $s  
}  
  
read -p "输入第一个数" first  
read -p "输入第二个数" second  
  
-----  
  
$ sum1 $first $second
```

2.第二种方法

```
sum2() {  
    s=${$1 + $2}  
    echo $s  
}  
  
-----  
  
$ sum2 $1 $2  
  
#$1 $2 代表执行脚本函数时命令后面跟的两个参数位置参数
```



函数的调用

- 函数的调用方式

- ❑ 可在交互式环境下定义函数
- ❑ 可将函数放在脚本文件中作为它的一部分
- ❑ 可放在只包含函数的单独文件中

● 调用：直接执行函数名即可，不需要带小括号。函数名出现的地方，会被自动替换为函数代码



第9章 Shell编程

```
$ vim 1.sh
```

```
-----  
#!/bin/bash
```

```
#定义函数
```

```
sum_fun() {
```

```
    #函数体内部的$1 $2 代表的是调用函数时，函数后面跟的位置参数
```

```
    echo $sum
```

```
    echo "在函数体内部的\${#}代表调用函数时，函数后面跟的参数个数，当前>函数后面有${#}个参数"
```

```
    echo "在函数体内部的${@}代表调用函数时，函数后面跟的所有参数，当前>函数后面的参数有：${@}"
```

```
    echo "在函数体内部， ${0}代表当前脚本$0"
```

```
}
```

```
-----  
#调用函数
```

```
#函数体外的$1 $2 代表的是执行脚本时，脚本后面跟的位置参数
```

```
$ sum2 $1 $2
```



第9章 Shell编程

- 使用函数递归目录/var/log，如果是文件，直接输出文件名，如果是目录，则输出目录名。

```
$ vim 4.sh
```

```
-----  
#!/bin/bash
```

```
listf () {  
    for f in $(ls $1)  
    do  
        if [ -d "$1/$f" ]; then  
            echo "$2目录$f"  
        else  
            echo "$2文件$f"  
        fi  
    done  
}
```

```
-----  
$ listf $(/var/log) $()
```

```
$ listf "/var/log" $()
```



第9章 Shell编程

- 通过脚本输出环境变量PATH所包含的所有目录以及其中的子目录和所有可执行文件;

```
$ vim 5.sh
```

```
-----  
#!/bin/bash
```

```
list_path() {  
    IFSB=$IFS  
    IFS=$IFS':'  
    for F in $PATH  
    do  
        echo "$F"  
    done  
    IFS=$IFS  
}
```

```
#定义一个递归函数
```

```
listf() {  
    for F in $(ls $1)  
    do  
        if [ -d "$1/$F" ];then  
            echo "$2$F"
```

```
        listf "$1/$F" " $2"  
    else  
        if [ -x "$1/$F" ];then  
            echo "$2$F"  
        fi  
    fi  
done  
}
```

```
#获取PATH变量的所有目录列表
```

```
folder=$(list_path)
```

```
#遍历PATH变量的目录，并用函数递归查询
```

```
for path in $folder  
do  
    echo $path  
    listf "$path" " "  
done
```



函数库文件

- 可以将经常使用的函数存入一个单独的函数文件，然后将函数文件载入shell，再进行调用函数。
- 在shell脚本或交互式shell中调用函数文件, 格式如下:

```
$ ./filename  
$ source filename
```



第9章 Shell编程

```
$ vim cla.sh
```

```
-----  
#!/bin/bash  
sum_fun() {  
    echo ${1 + $2}  
}  
sub_fun() {  
    echo ${1 - $2}  
}  
multi_fun() {  
    echo ${1 * $2}  
}  
divi_fun() {  
    if [ $2 -eq 0 ];then  
        echo "除数不可为0"  
    else  
        echo ${1 / $2}  
    fi  
}
```



```
$ vim test.sh
```

```
-----  
#!/bin/bash  
#加载函数库文件到当前脚本的shell  
source /home/ubuntu/clr.sh  
  
value1=10  
value2=5  
  
res1=$(sum_fun $value1 $value2)  
res2=$(sub_fun $value1 $value2)  
res3=$(multi_fun $value1 $value2)  
res4=$(divi_fun $value1 $value2)  
  
echo "加法的结果为$res1"  
echo "减法的结果为$res2"  
echo "乘法的结果为$res3"  
echo "除法的结果为$res4"
```



第9章 Shell编程